Category	Method	Edit Area	Edit Function	No Training	Batch Edit	Edited #Params
Recogintion	MemPrompt [154]	memory+retriever	Input $\rightarrow$ [Mem : Input]	✔	✔	–
Phase	SERAC [153]	memory+classifier +auxiliary model	Output $\rightarrow$ Model$_{cf}(\boldsymbol{x})$	✗	✔	–
	MeLLo [155]	memory+retriever	Input $\rightarrow$ [Mem : Input]	✔	✗	–
	IKE [156]	memory+retriever	Input $\rightarrow$ [Mem : Input]	✔	✗	–
	ICE [157]	prompt	Input $\rightarrow$ [Mem : Input]	✔	✗	–
	PokeMQA [158]	memory+retriever	Input $\rightarrow$ [Mem : Input]	✗	✗	–
Association	Language Patches[159]	Output head + params	$\boldsymbol{h} \rightarrow \lambda \boldsymbol{h} +$ $(1-\lambda)$Patch$(\boldsymbol{x})$	✔	✔	$d_h \times$ #Output
Phase	CaliNET [160]	FFN+params	$\boldsymbol{h} \rightarrow \boldsymbol{h} +$ FFN$_{\text{add}}(\boldsymbol{x})$	✗	✔	$N \times d_h$
	T-Patcher[161]	FFN+params	$\boldsymbol{h} \rightarrow \boldsymbol{h} +$ FFN$_{\text{add}}(\boldsymbol{x})$	✗	✗	$N \times d_h$
	REMEDI [162]	auxiliary model	$\boldsymbol{h} \rightarrow$ REMEDI$(\boldsymbol{x})$	✗	✗	$d_h \times d_h$
	GRACE [163]	FFN+codebook	$\boldsymbol{h} \rightarrow$ GRACE$(\boldsymbol{x})$	✗	✗	$N \times 2d_h$
	LoRA [164]	Attn or FFN	$\boldsymbol{h} \rightarrow \boldsymbol{h} + s \cdot$ LoRA$(\boldsymbol{x})$	✗	✔	$2L \times 2d_{am}d_h$
	MELO [165]	Attn or FFN	$\boldsymbol{h} \rightarrow \boldsymbol{h} + s \cdot$ LoRA$(\boldsymbol{x})$	✗	✗	$2L \times 2d_{am}d_h$
Mastery	FT-Constrained [166]	Any	$\boldsymbol{W} \rightarrow \boldsymbol{W}'$	✗	✔	$2 \times L \times d_m d_h$
Phase	ENN [167]	Any	$\boldsymbol{W} \rightarrow \boldsymbol{W}'$	✗	✔	$2 \times L \times d_m d_h$
	KE[168]	Attn or FFN +auxiliary model	$\boldsymbol{W} \rightarrow \boldsymbol{W}'$	✗	✔	$2 \times L \times d_m d_h$
	SLAG [169]	Attn or FFN +auxiliary model	$\boldsymbol{W} \rightarrow \boldsymbol{W}'$	✗	✔	$2 \times L \times d_m d_h$
	MEND [170]	FFN+ auxiliary model	$\boldsymbol{W} \rightarrow \boldsymbol{W}'$	✗	✔	$2 \times L \times d_m d_h$
	KN [39]	FFN	$\boldsymbol{W}_{\text{down}} \rightarrow \boldsymbol{W}'_{\text{down}}$	✔	✗	$L \times N \times d_h$
	ROME [79]	FFN	$\boldsymbol{W}_{\text{down}} \rightarrow \boldsymbol{W}'_{\text{down}}$	✔	✗	$d_m d_h$
	MEMIT [171]	FFN	$\boldsymbol{W}_{\text{down}} \rightarrow \boldsymbol{W}'_{\text{down}}$	✔	✔	$L \times d_m d_h$
	PMET [172]	FFN	$\boldsymbol{W}_{\text{down}} \rightarrow \boldsymbol{W}'_{\text{down}}$	✔	✔	$L \times d_m d_h$
	MALMEN [173]	FFN	$\boldsymbol{W}_{\text{down}} \rightarrow \boldsymbol{W}'_{\text{down}}$	✗	✔	$L \times d_m d_h$
	BIRD [174]	FFN	$\boldsymbol{W}_{\text{down}} \rightarrow \boldsymbol{W}'_{\text{down}}$	✔	✗	$d_m d_h$
	AlphaEdit [175]	FFN	$\boldsymbol{W}_{\text{down}} \rightarrow \boldsymbol{W}'_{\text{down}}$	✔	✔	$L \times d_m d_h$

Table 2: Comparison between representative approaches of knowledge editing for LLMs. **No Training** refers to the methods that do not require additional training; **Batch Edit** means whether the methods can support editing multiple cases simultaneously in just one process. **Edit Area** refers to where the model's components are used; **Editor #Params** indicates the parameters that need to be updated for editing. L refers to the number of layers to update. d_h denotes the dimensionality of the hidden layers in the Transformers. d_m refers to the intermediate dimension that exists between the up projection and the down projection. N symbolizes the total number of neurons that undergo updates within each individual layer.

3.3.1 Recognition Phase: Resorting to External Knowledge

When humans encounter new information, we do not always master it immediately. Instead, with the right context and examples, we can process and reason through this new knowledge. LLMs exhibit a similar capacity for in-context learning. This kind of method usually maintains a memory M and retrieves the most relevant cases for each input. IKE [156] exemplifies this approach by constructing three types of demonstrations – copy, update, and retain – to aid the model in producing reliable fact editing. It utilizes a demonstration store, formed from training sets, to guide the model towards generating the appropriate answer by retrieving the most pertinent demonstrations. Meanwhile, as a simple change in knowledge would lead to ripple effects [157], MeLLo [155] decomposes the question into different sub-questions for tackling multi-hop questions and retrieves the updated fact from the memory for each sub-question. Building on this, PokeMQA [158] offers a more robust method for question decomposition, introducing a programmable scope detector and knowledge prompts for enhanced reliability.

Humans also often utilize tools to augment their learning and problem-solving abilities. Likely, SERAC [153] builds a new counterfact model by retaining the new model and adopting a classifier to determine whether to use the counterfact model to answer the question. This method is straightforward and practically applicable, requiring no alterations to the original model. It's particularly advantageous for real-world use, given its ease of implementation. However, it's important to note that this approach can be vulnerable to issues such as retrieval errors (*e.g.*noise [176], harmful content [177]) and knowledge conflict problems [178, 179]. Recently, Yu et al. [180] investigats various scenarios in which language models opt for either the in-context answer or the memorized answer.

This research sheds light on the potential application of the method mentioned earlier, as it may offer insights into when and how to utilize it.

3.3.2 Association Phase: Merge the Knowledge into the Model

Unlike the recognition phase, this kind of method learns a representation for the new knowledge $\boldsymbol{h}_{\text{Know}}$ and merges this information with the original model's representation $\boldsymbol{h}$.

Murty et al. [159] proposes a knowledge patch as a new output head and interpolates the new head with the original head. Specially, inspired by previous findings that FFN may store knowledge, several methods integrate the knowledge into the FFN part. These methods add the neuron to the FFN and after the edit, the output is a combination of the previous FFN's output and the newly added knowledge:

$$\text{FFN}^{'}(\mathbf{x}) = \text{FFN}(\mathbf{x}) + \triangle\text{FFN}(\mathbf{x}), \tag{8}$$

In particular, T-Patcher [161] adds one neuron for each output error, while CaliNet [160] adds the knowledge via a fixed number of neurons. Meanwhile, Wu et al. [164] adopts LoRA to conduct knowledge edits. LoRA is a parameter-efficient fine-tuning method that freezes the weights of the LLM and introduces trainable rank decomposition matrices into the Transformer layers during the fine-tuning process. Hence, the $\boldsymbol{h}_{\text{Know}}$ is $\boldsymbol{x}\boldsymbol{W}_{\text{down}}\boldsymbol{W}_{\text{up}}$. Based on this, MELO [165] suggests a plug-in model editing method that uses dynamic LoRA to change the way language models work by indexing LoRA blocks dynamically based on an internal vector database. Instead of adding parameters to the model, REMEDI [162] directly substitutes the representation of the entity h_{entity} by incorporating an attribute vector h_{attr} into its original model's representation. Specifically, it learns the updated hidden states using an affine transformation $h_{\text{entity}} + Wh_{\text{attr}} + b$ and replaces the LM's entity representation with it. In contrast, GRACE [163] adopts a unique approach by maintaining a discrete codebook that functions as an *Adapter*. This codebook is dynamically updated over time, allowing for the modification and refinement of a model's predictions. When the model encounters the knowledge for editing, it searches the codebook and replaces the hidden states as the value in the codebook. Overall, we can use a mathematical formula to represent these methods uniformly:

$$\boldsymbol{h}_{final} = \boldsymbol{h} + \boldsymbol{h}_{\text{know}} \tag{9}$$

This kind of method merged the information with the original model, making the weighting of knowledge from different sources a crucial parameter to consider. Given that these information sources often differ and may even conflict, the issue of knowledge conflict, as highlighted in Wang et al. [178], remains a significant challenge. To address this issue, F-Learning [181] introduces a "forgetting before learning" paradigm to achieve forgetting of old knowledge and learning of new knowledge based on parametric arithmetic. Additionally, determining the optimal point of integration for this information within the model is a critical aspect of this method. It is not just about merging the information, but also about where in the model's structure this integration occurs for maximum effectiveness and minimal disruption. Furthermore, the capacity of the model's parameters to store this integrated information is an area that still requires exploration. If every piece of edited knowledge necessitates additional parameters, the model's parameter could increase significantly with each edit. This raises concerns about scalability and efficiency, as continuously expanding the number of parameters might lead to issues like increased computational requirements.

3.3.3 Mastery Phase: Editing Intrinsic Knowledge

Despite the success of the previous two kinds of methods, we still confront how the model stores the knowledge and how they utilize and express the knowledge. Here, we come to the most important part of knowledge editing: the mastery stage. In this part, the model is required to learn the knowledge of its own parameters and master the knowledge by itself. Fine-tuning the model is the direct way to update the knowledge; however, training the whole model requires enormous computational resources and is time-consuming. Meanwhile, the finetuning technique usually suffers from catastrophic forgetting and overfitting. Constrained Fintune [166] utilizes a regularization to help the model keep the unrelated knowledge. Currently, many researchers endeavor to use knowledge-specific methods to modify the $\Delta\boldsymbol{W}$. These methods can be classified into two categories: meta-learning and locate-and-edit.

Meta Learning To overcome these drawbacks, some meta-learning methods are proposed to edit the model. Instead of updating the weights directly, this kind of method teaches a hypernetwork to learn the change $\Delta \boldsymbol{W}$ of the model. KE [168] directly uses the representation of the new knowledge to train the model to update the matrix. SLAG [169] introduces a new training objective considering sequential, local, and generalizing model updates. The $\Delta \boldsymbol{W}$ in these methods has the same dimensions as the model's matrix. In order to overcome it, MEND [170] applies the rank-one decomposition to divide the model into two rank-one matrices, from which it is possible to compute the $\Delta \boldsymbol{W}$, significantly reducing the number of parameters. While these methods have shown some promising results, they fail on multi-edits as they ignore the conflicts between these edits. Han et al. [182] proposes a novel framework to divide-and-conquer edits with parallel editors. Specifically, they design explicit multi-editor **MoEditor** and implicit multi-editor **ProEditor** to learn diverse editing strategies in terms of dynamic structure and dynamic parameters, respectively, which allows solving the conflict data in an efficient, end-to-end manner. Also, MALMEN [173] improves MEND by formulating the parameter shift aggregation as a least squares problem and supports massive editing simultaneously.

Location-then-Edit Despite the effectiveness of previous work, how the LLMs store this knowledge is still unknown. Some work [41, 42, 97], has learned the mechanism of LLMs knowledge and found that the knowledge was stored in the FFN . Based on these works, some conduct knowledge editing by first locating where the knowledge was stored and then editing the specific area. Knowledge Neuron [39] proposed a knowledge attribution method by computing the sensitivity of the gradient change. They then directly modify the corresponding value slots using the embedding of the target knowledge. ROME [79] and MEMIT [171] employ a causal analysis method to detect which part of hidden states plays more importance. They view the editing as a minimum optimization and edit the weights. Despite the effectiveness of editing the FFN area, PMET [172] also conducts editing via the attention head and demonstrates a better performance. BIRD [174] proposes bidirectionally inverse relationship modeling. They designed a set of editing objectives that incorporate bidirectional relationships between subject and object into the updated model weights and demonstrate the effectiveness of alleviating the reverse curse [183] of the knowledge learning. To more effectively address the disruption of originally preserved knowledge within Large Language Models (LLMs), AlphaEdit [175] proposes an innovative approach. This method involves projecting perturbations into the null space of the preserved knowledge prior to their application to model parameters, thereby substantially reducing the issue.

This kind of method, which directly edits a model's parameters, offers a more permanent solution for altering its behavior. The changes are embedded into the model's structure, so they cannot be circumvented even if a user has access to the model's weights. This ensures lasting and reliable modifications. However, the side effects are not under control since the mechanism of LLMs is unclear. Some researchers are skeptical about this kind of method [184], so it is still a premature research area that requires further investigation.

3.4 New Benchmark: KnowEdit

To evaluate the effectiveness of knowledge editing methods, several datasets have been proposed. In this Section, we present an overview of the current datasets used for knowledge editing and introduce a new benchmark, **KnowEdit**[3], which serves as a comprehensive evaluation framework for various knowledge editing techniques.

Task	**Knowledge Insertion**	**Knowledge Modification**				**Knowledge Erasure**
Datasets	**WikiData**$_{recent}$	**ZsRE**	**WikiBio**	**WikiData**$_{counterfact}$	**Convsent**	**Sanitation**
Type	Fact	Question Answering	Hallucination	Counterfact	Sentiment	Unwanted Info
# Train	570	10,000	592	1,455	14,390	80
# Test	1,266	1230	1,392	885	800	80

Table 3: Statistics on the benchmark **KnowEdit**, with six selected datasets for the evaluation of knowledge editing methods. We select different knowledge types for the insertion, modification, and erasure settings.

[3] https://huggingface.co/datasets/zjunlp/KnowEdit.